

Año de la Esperanza y el Fortalecimiento de la Democracia”

FACULTAD DE INGENIERÍA
ESCUELA PROFESIONAL DE INGENIERÍA SISTEMAS E INFORMATICA



**SISTEMA DE GESTIÓN DE INVENTARIO Y VENTAS PARA UNA
LIBRERÍA**

DOCENTE:

Yesenia Concha Ramos

NRC:

29616

INTEGRANTES:

Ortiz Gutierrez Nasdig Jordan 100%

SEMESTRE

2026-10 CUSCO

PERÚ

SISTEMA DE GESTIÓN DE INVENTARIO Y VENTAS PARA UNA LIBRERÍA

CAPÍTULO 1: ANÁLISIS DEL PROBLEMA	5
1.1 Descripción del problema	5
1.2 Requerimientos del sistema	6
1.3 Estructuras de datos propuestas	8
1.4 Justificación de la elección	9
CAPÍTULO 2: DISEÑO DE LA SOLUCIÓN	10
2.1 Descripción de estructuras de datos y operaciones	10
2.2 Algoritmos principales	12
2.3 Diagramas de flujo	18
2.4 Justificación del diseño	23
CAPÍTULO 3: SOLUCIÓN FINAL	24
3.1 Código limpio, comentado y estructurado	24
3.2 Capturas de pantalla de las pruebas realizadas	43
3.3 Manual de usuario	49
CAPÍTULO 4: EVIDENCIAS DE TRABAJO EN EQUIPO	54
4.1 Repositorio con control de versiones	54
4.2 Plan de trabajo y roles asignados	55

ÍNDICE DE TABLAS

Tabla 1. Relación entre estructuras y problema resuelto	9
Tabla 2. Operaciones implementadas por estructura de datos	12

Tabla 3. Distribución de responsabilidades	55
--	----

Tabla 4. Cronograma de trabajo	56
--------------------------------------	----

ÍNDICE DE FIGURAS

Figura 1. Diagrama de flujo de Registrar Libro	18
--	----

Figura 2. Diagrama de flujo de Eliminar Libro	19
---	----

Figura 3. Diagrama de flujo de Registrar Reposición	20
---	----

Figura 4. Diagrama de flujo de Atender Reposición	21
---	----

Figura 5. Diagrama de flujo de Registrar Acción	22
---	----

Figura 6. Diagrama de flujo de Eliminar Última Acción	23
---	----

Figura 7. Imagen Librerías	24
----------------------------------	----

Figura 8. Estructuras	25
-----------------------------	----

Figura 9. Variables globales y punteros principales	26
---	----

Figura 10. Categorías predefinidas	26
--	----

Figura 11. Código elegir categoría	27
--	----

Figura 12. Código limpiar buffer	27
--	----

Figura 13. Código para minúsculas	28
---	----

Figura 14. Solo números o texto válido	28
--	----

Figura 15. Pedir texto sin errores	29
--	----

Figura 16. Pedir stock sin negativos	29
--	----

Figura 17. Pedir precio mayor a cero	30
--	----

Figura 18. Agregar al historial	30
---------------------------------------	----

Figura 19. Ver la última acción	31
---------------------------------------	----

Figura 20. Quitar la última acción	31
Figura 21. Declaración adelantada de buscarNodo	32
Figura 22. Registrar un ticket de reposición	32
Figura 23. Atender el primer ticket pendiente	33
Figura 24. Buscar un libro en la lista	33
Figura 25. Mostrar los datos de un libro	34
Figura 26. Agregar un libro al inventario	34
Figura 27. Encontrar un libro por título	35
Figura 28. Actualizar datos de un libro	36
Figura 29. Quitar un libro del inventario	37
Figura 30. Ver todos los libros registrados	38
Figura 31. Guardar el inventario en disco	39
Figura 32. Cargar el inventario al iniciar	40
Figura 33. Menú del sistema	41
Figura 34. Función principal del programa	42
Figura 35. Registro exitoso de un libro	43
Figura 36. Búsqueda de un libro registrado	44
Figura 37. Modificación de información de un libro	45
Figura 38. Visualización del inventario	46
Figura 39. Registro de solicitud de reposición	47
Figura 40. Consulta del historial de acciones	48
Figura 41. Validación de datos incorrectos	49
Figura 42. Historial de commits del proyecto	54

CAPÍTULO 1: ANÁLISIS DEL PROBLEMA

1.1 Descripción del problema

Cuando una librería inicia sus actividades, normalmente el control del inventario puede realizarse de forma manual debido a la poca cantidad de libros disponibles. Sin embargo, conforme el negocio crece y aumenta la variedad de títulos, géneros y autores, comienza a ser más difícil administrar toda la información únicamente mediante registros manuales o la memoria de los trabajadores.

Uno de los problemas más comunes es no conocer con exactitud qué libros se encuentran disponibles, cuántas unidades quedan en stock o cuáles requieren reposición. Esto puede ocasionar errores en la atención al cliente, pérdida de tiempo al buscar información y dificultades para mantener actualizado el inventario.

Además, cuando se realizan cambios frecuentes en los registros, como la actualización de stock o la modificación de precios, resulta complicado recordar qué acciones fueron realizadas recientemente y quién las ejecutó. La ausencia de un control organizado puede generar inconsistencias en la información almacenada.

Por otra parte, cuando un libro se agota, es importante contar con un mecanismo que permita identificar rápidamente aquellos productos que necesitan ser repuestos para evitar afectar las ventas futuras.

Para solucionar estos problemas se propone desarrollar un sistema de gestión de inventario para una librería utilizando estructuras de datos dinámicas en lenguaje C++. El sistema permitirá registrar libros, buscar información, modificar registros, controlar solicitudes de reposición y mantener un historial de acciones realizadas dentro del sistema.

El objetivo principal es mejorar la organización del inventario, reducir errores humanos y facilitar la administración de la información mediante el uso de listas enlazadas, colas y pilas.

1.2 Requerimientos del sistema

Requerimientos funcionales

Registrar libros

El sistema deberá permitir registrar nuevos libros dentro del inventario. Para ello se almacenará información relevante como título, autor, categoría, precio y cantidad disponible. Esta operación permitirá mantener actualizado el catálogo de productos disponibles en la librería.

Buscar libros por título

El sistema deberá permitir localizar un libro específico utilizando su título como criterio de búsqueda. Esto facilitará la consulta rápida de información sin necesidad de revisar manualmente todo el inventario.

Eliminar libros

Permite retirar permanentemente un libro del inventario cuando ya no se comercializa o cuando fue registrado por error.

Modificar información de libros

El sistema deberá permitir actualizar los datos de un libro previamente registrado. Esto incluye cambios en el precio, la categoría o la cantidad disponible en stock. Esta funcionalidad permitirá mantener la información actualizada conforme cambien las necesidades del negocio.

Mostrar inventario completo

El sistema deberá permitir visualizar todos los libros registrados junto con su información correspondiente. Esta operación facilitará el control general del inventario y permitirá verificar el estado actual de los registros almacenados.

Registrar solicitudes de reposición

El sistema deberá permitir registrar aquellos libros que requieran ser repuestos debido a una disminución significativa del stock o al agotamiento total de existencias. Esto ayudará a mantener el inventario abastecido.

Atender solicitudes de reposición

El sistema deberá permitir procesar las solicitudes de reposición registradas previamente. Una vez atendida una solicitud, esta será retirada de la estructura correspondiente para mantener actualizada la información.

Registrar acciones realizadas

El sistema deberá almacenar un historial de acciones importantes realizadas por el usuario, como registros o modificaciones de libros. Esto permitirá llevar un control básico de las actividades ejecutadas dentro del sistema.

Consultar la última acción registrada

El sistema deberá permitir visualizar la acción más reciente almacenada en el historial. Esta funcionalidad facilitará el seguimiento de los últimos cambios realizados sobre el inventario.

Requerimientos no funcionales

Desarrollo en lenguaje C++

El sistema deberá ser implementado utilizando el lenguaje de programación C++, ya que es el lenguaje empleado durante el curso y permite trabajar adecuadamente con estructuras de datos dinámicas.

Uso de estructuras dinámicas

El sistema deberá implementar listas enlazadas, colas y pilas para la gestión de la información. De esta manera se aplicarán los conocimientos desarrollados durante la asignatura.

Funcionamiento mediante consola

La interacción con el usuario se realizará a través de una interfaz de consola. Esto permitirá centrar el desarrollo en la lógica del sistema y en la implementación de las estructuras de datos.

Organización clara de la información

La información deberá mantenerse organizada de forma que pueda ser consultada y modificada fácilmente. Esto facilitará tanto el uso del sistema como su mantenimiento posterior.

Facilidad de mantenimiento

El código deberá desarrollarse de forma estructurada y modular para facilitar futuras correcciones, ampliaciones o mejoras.

Posibilidad de ampliación

El diseño deberá permitir incorporar nuevas funcionalidades en el futuro sin necesidad de rehacer completamente el sistema.

Facilidad de uso

El sistema deberá presentar mensajes claros y comprensibles para que cualquier usuario pueda utilizar sus funciones básicas sin dificultad.

1.3 Estructuras de datos propuestas

Lista enlazada

La lista enlazada será utilizada para almacenar el inventario completo de libros. Cada libro será almacenado dentro de un nodo que contendrá su información y una referencia al siguiente elemento.

Esta estructura permitirá que el inventario pueda crecer dinámicamente sin necesidad de definir previamente una cantidad máxima de registros.

Cola

La cola será utilizada para administrar los libros pendientes de reposición.

Debido a que una cola trabaja bajo el principio FIFO (First In, First Out), permitirá que los primeros libros registrados como pendientes sean también los primeros en ser atendidos.

Pila

La pila será utilizada para almacenar el historial de acciones realizadas dentro del sistema.

Al seguir el principio LIFO (Last In, First Out), permitirá consultar rápidamente la acción más reciente realizada por el usuario.

Tabla 1. Relación entre estructuras y problema resuelto

Estructura	Problema que resuelve
Lista enlazada	Almacenar y administrar el inventario de libros.
Cola	Gestionar solicitudes de reposición.
Pila	Mantener historial de acciones recientes.

Fuente: Elaboración propia.

1.4 Justificación de la elección

Las estructuras seleccionadas fueron elegidas debido a que representan adecuadamente las necesidades del sistema.

La lista enlazada permite administrar una cantidad variable de libros sin establecer límites fijos de almacenamiento. Esto resulta conveniente porque el inventario puede aumentar o disminuir con el tiempo.

La cola fue seleccionada para gestionar solicitudes de reposición debido a que respeta el orden en que estas son registradas, permitiendo atender primero las solicitudes más antiguas.

La pila fue elegida para almacenar el historial de acciones porque normalmente interesa conocer primero la acción más reciente realizada dentro del sistema.

Estas estructuras permiten desarrollar una solución organizada, sencilla de implementar y alineada con los temas estudiados durante el curso.

CAPÍTULO 2: DISEÑO DE LA SOLUCIÓN

2.1 Descripción de estructuras de datos y operaciones

Lista enlazada (Inventario de libros)

La lista enlazada será la estructura principal del sistema. En ella se almacenarán todos los libros registrados dentro del inventario.

Operaciones de la lista enlazada

Registrar libro

Permite agregar un nuevo libro al inventario creando un nuevo nodo dentro de la lista.

Buscar libro

Permite localizar un libro recorriendo la lista hasta encontrar el título solicitado.

Eliminar libro

Permite localizar un libro dentro de la lista enlazada y eliminar su nodo correspondiente, liberando el espacio utilizado y actualizando los enlaces entre nodos para mantener la integridad de la estructura.

Modificar libro

Permite actualizar información previamente registrada, como stock, precio o categoría.

Mostrar inventario

Permite visualizar todos los libros almacenados dentro del sistema.

Cola (Reposición de libros)

La cola será utilizada para administrar los libros que requieren reposición.

Operaciones de la cola

Registrar reposición (Encolar)

Permite agregar un libro al final de la cola cuando necesita ser repuesto.

Atender reposición (Desencolar)

Permite retirar el primer libro registrado dentro de la cola indicando que la reposición ha sido atendida.

Pila (Historial de acciones)

La pila almacenará las acciones realizadas dentro del sistema.

Operaciones de la pila

Registrar acción (Push)

Permite almacenar una nueva acción en la parte superior de la pila.

Consultar última acción (Peek)

Permite visualizar la acción más reciente registrada.

Eliminar última acción (Pop)

Permite retirar la acción más reciente almacenada en la pila.

Tabla 2. Operaciones implementadas por estructura de datos

Estructura	Operación 1	Operación 2	Operación 3
Lista enlazada	Registrar libro	Buscar libro	Modificar libro
Lista enlazada	Mostrar inventario	Eliminar libro	—
Cola	Registrar reposición (Encolar)	Atender reposición (Desencolar)	—
Pila	Registrar acción (Push)	Consultar última acción (Peek)	Eliminar última acción (Pop)

Fuente: Elaboración propia.

2.2 Algoritmos principales

Pseudocódigo para Listas

- Registrar Libro

La lista enlazada almacena todos los libros del inventario. Para registrar un libro se solicitan los datos al usuario, se validan y luego se verifica si ya existe un libro con el mismo título. Si no existe, se crea un nuevo nodo y se inserta al inicio de la lista.

Proceso RegistrarLibro

Leer titulo

Leer autor

Leer categoria

Leer stock

Leer precio

Si titulo esta vacio O autor esta vacio Entonces

 Escribir "Datos invalidos"

Sino

 actual <- cabezaLista

```

encontrado <- Falso
Mientras actual <> Nulo Hacer
    Si Minusculas(actual.titulo) = Minusculas(titulo) Entonces
        encontrado <- Verdadero
    FinSi
    actual <- actual.siguiente
FinMientras
Si encontrado Entonces
    Escribir "El libro ya existe"
Sino
    Crear nuevoNodo
    nuevoNodo.titulo <- titulo
    nuevoNodo.autor <- autor
    nuevoNodo.categoria <- categoria
    nuevoNodo.stock <- stock
    nuevoNodo.precio <- precio
    nuevoNodo.siguiente <- cabezaLista
    cabezaLista <- nuevoNodo
    Escribir "Libro registrado correctamente"
FinSi
FinSi
FinProceso

```

- Eliminar Libro

Para eliminar un libro se busca dentro de la lista enlazada. Si se encuentra, se ajustan los enlaces entre nodos para que el nodo eliminado deje de formar parte de la lista.

Proceso EliminarLibro

Leer tituloBuscado

actual <- cabezaLista

anterior <- Nulo

encontrado <- Falso

Mientras actual <> Nulo Y encontrado = Falso Hacer

 Si Minusculas(actual.titulo) = Minusculas(tituloBuscado) Entonces

 encontrado <- Verdadero

 Sino

 anterior <- actual

 actual <- actual.siguiente

 FinSi

FinMientras

Si encontrado Entonces

 Si anterior = Nulo Entonces

 cabezaLista <- actual.siguiente

 Sino

 anterior.siguiente <- actual.siguiente

 FinSi

 Eliminar actual

 Escribir "Libro eliminado"

Sino

 Escribir "Libro no encontrado"

FinSi

Fin Proceso

Pseudocódigo para Colas

- Registrar reposición

La cola almacena libros que necesitan reposición. Cada nuevo pedido se agrega al final de la cola para respetar el orden de llegada.

Proceso RegistrarReposicion

Leer tituloLibro

Buscar libro en la lista

Si libro no existe Entonces

 Escribir "El libro no existe"

Sino

 Crear nuevoNodoCola

 nuevoNodoCola.titulo <- tituloLibro

 nuevoNodoCola.siguiente <- Nulo

 Si frente = Nulo Entonces

 frente <- nuevoNodoCola

 final <- nuevoNodoCola

 Sino

 final.siguiente <- nuevoNodoCola

 final <- nuevoNodoCola

 FinSi

 Escribir "Reposicion registrada"

FinSi

FinProceso

- Atender reposición

La cola sigue el principio FIFO (First In First Out). El primer libro que ingresó a la cola será el primero en salir.

Proceso AtenderReposicion

Si frente = Nulo Entonces

 Escribir "No existen reposiciones pendientes"

Sino

 temporal <- frente

 frente <- frente.siguiente

Si frente = Nulo Entonces

 final <- Nulo

FinSi

Eliminar temporal

Escribir "Reposicion atendida"

FinSi

FinProceso

Pseudocódigo para Pilas

- Registrar acción

La pila almacena el historial de acciones realizadas en el sistema. Cada nueva acción se inserta en la parte superior de la pila.

Proceso RegistrarAccion

Leer descripcionAccion

Crear nuevoNodoPila

nuevoNodoPila.accion <- descripcionAccion

nuevoNodoPila.siguiente <- cima

cima <- nuevoNodoPila

FinProceso

- Eliminar última acción

La pila sigue el principio LIFO (Last In First Out). La última acción registrada será la primera en eliminarse.

Proceso EliminarUltimaAccion

Si cima = Nulo Entonces

 Escribir "No existen acciones registradas"

Sino

 temporal <- cima

 cima <- cima.siguiete

 Eliminar temporal

 Escribir "Ultima accion eliminada"

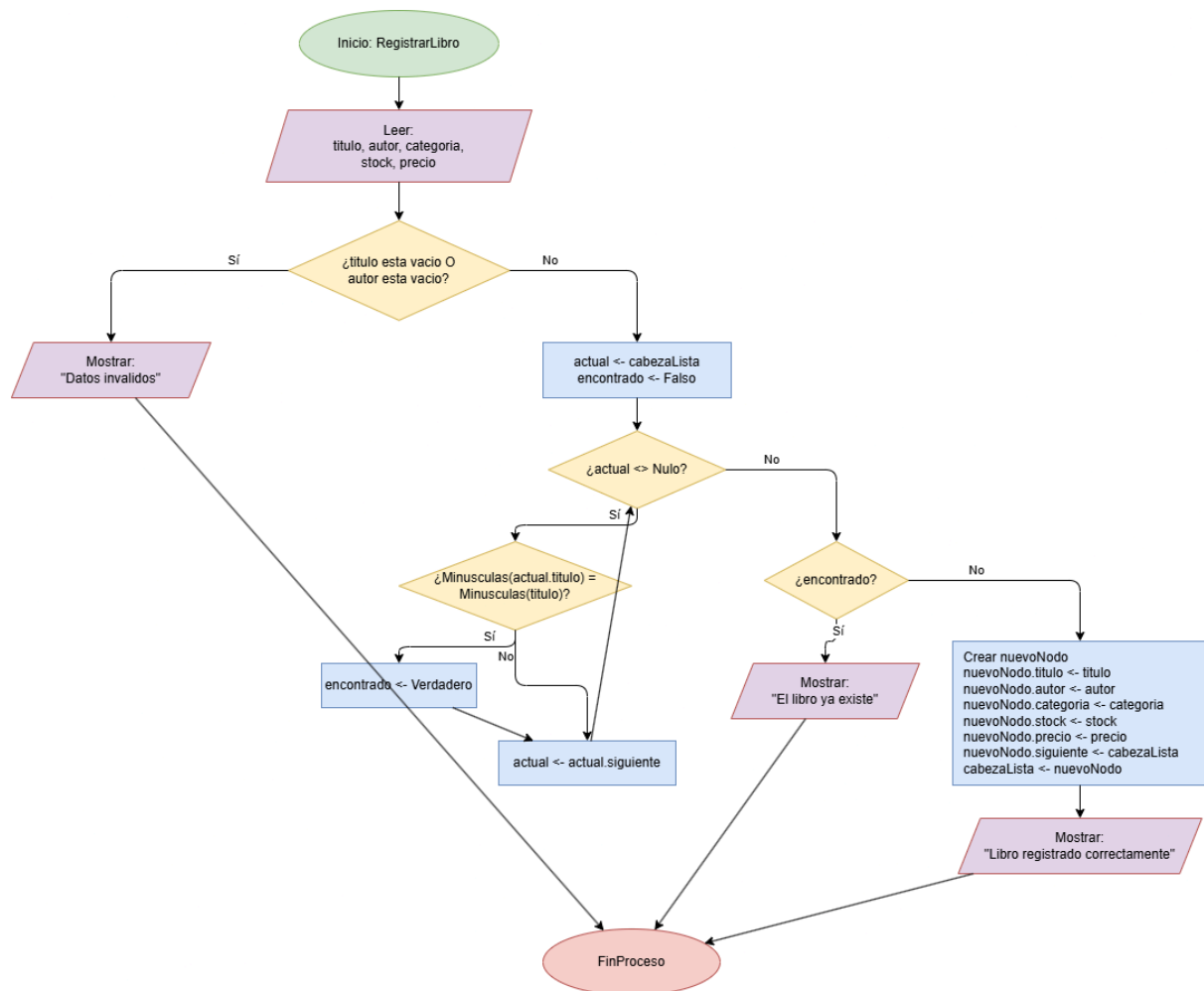
FinSi

FinProceso

2.3 Diagramas de flujo

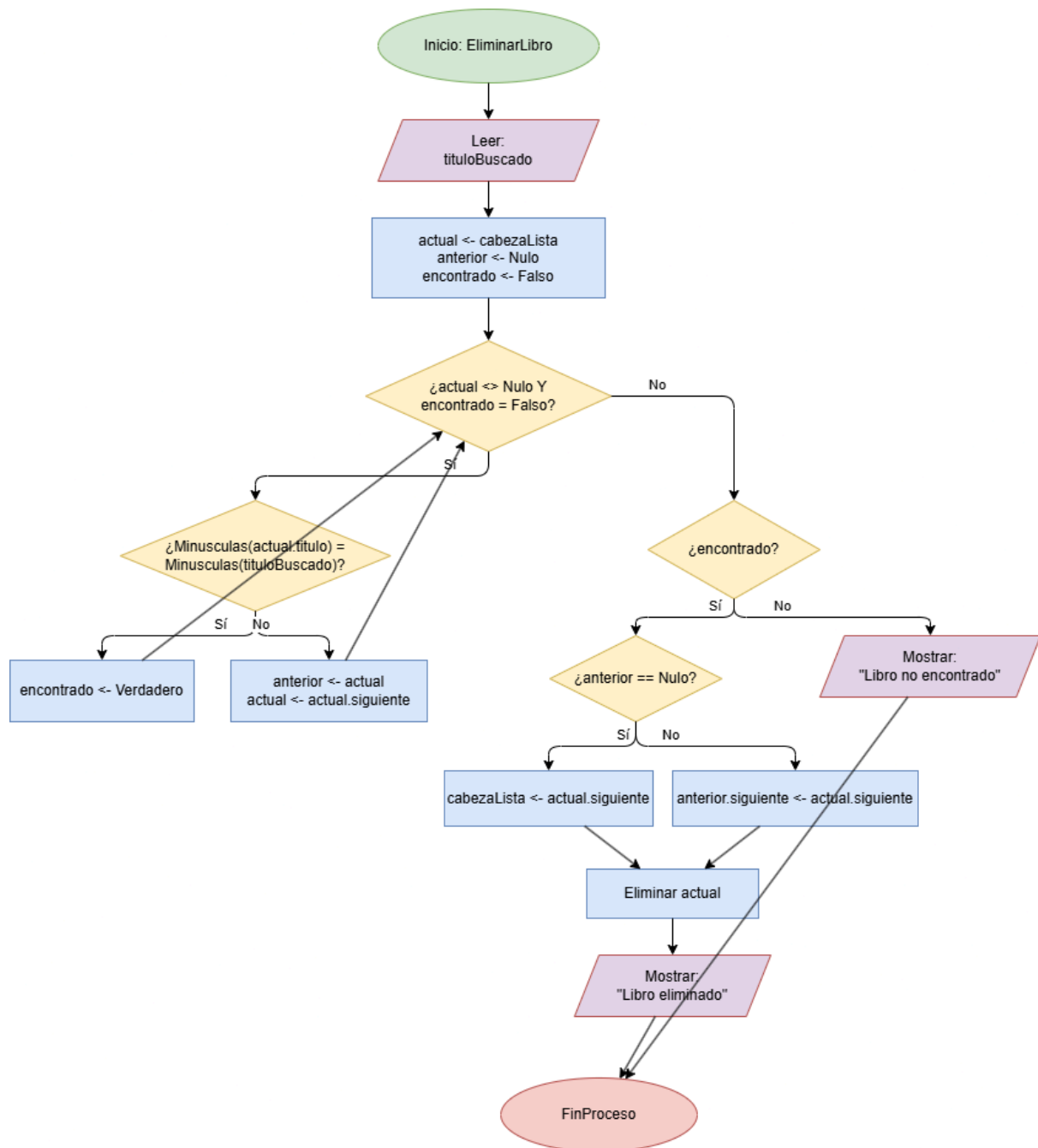
Listas

Figura 1. Diagrama de flujo de Registrar Libro



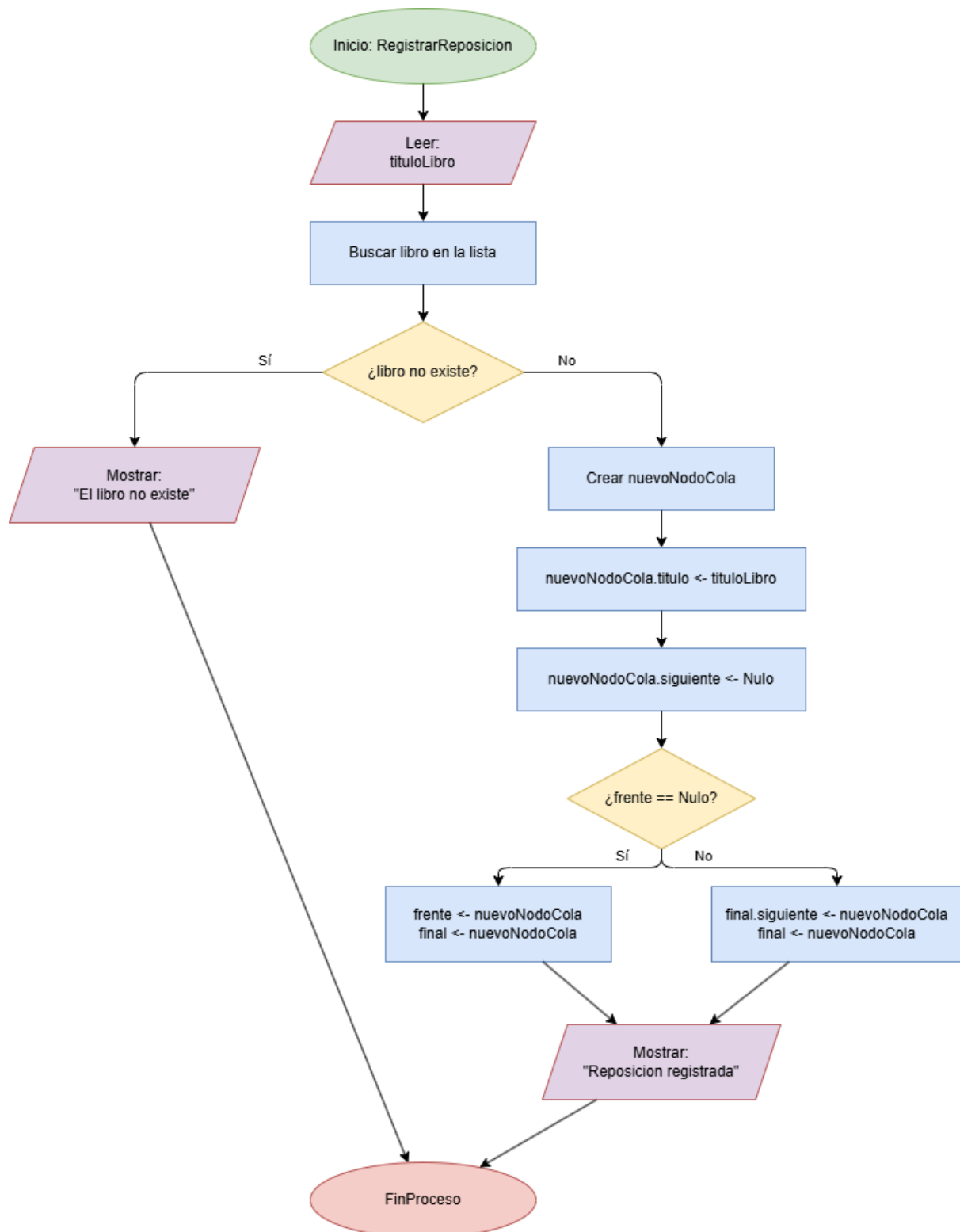
Nota: Diagrama de flujo generado en draw.io

Figura 2. Diagrama de flujo de Eliminar Libro



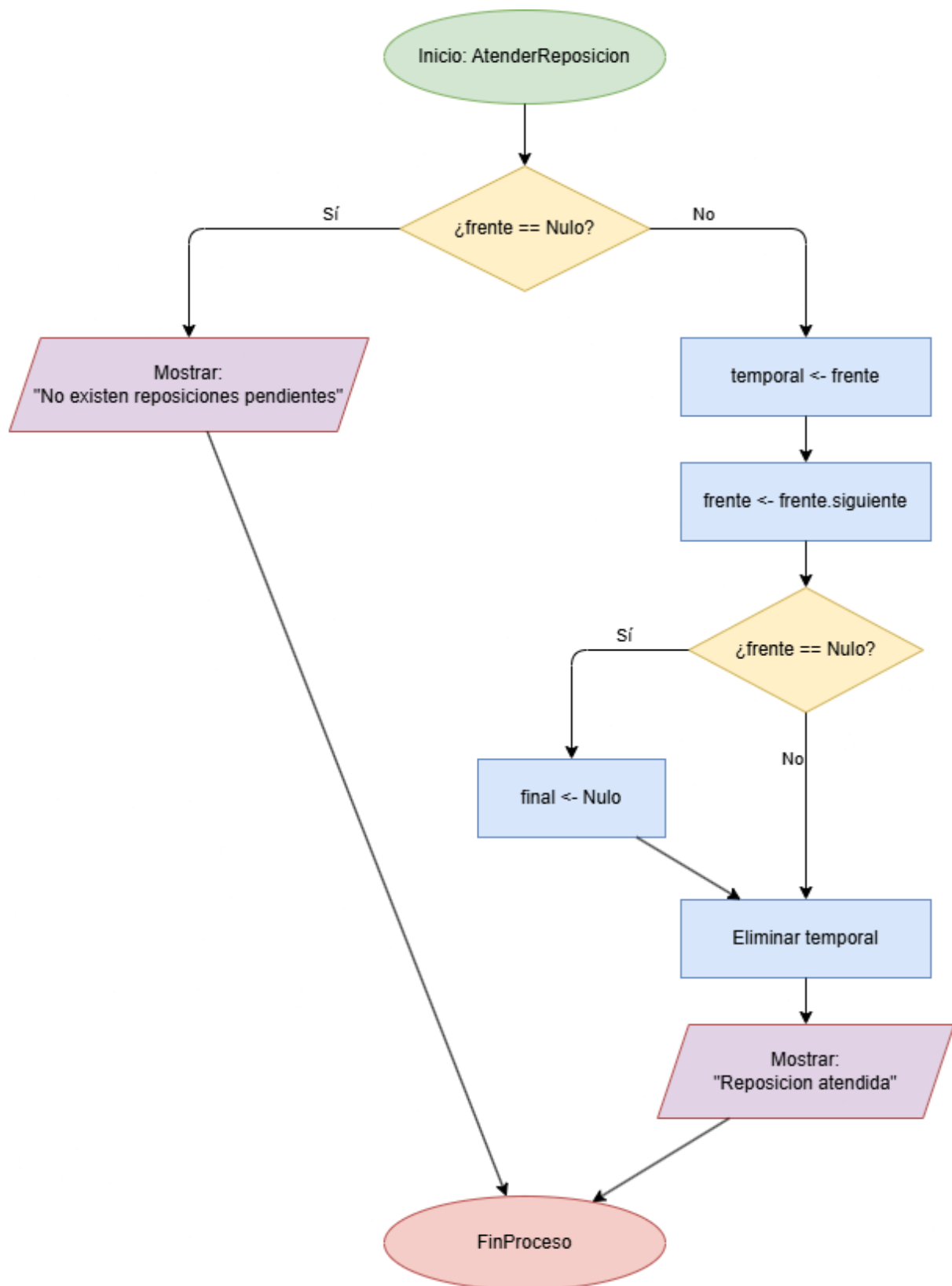
Nota: Diagrama de flujo generado en draw.io

Figura 3. Diagrama de flujo de Registrar Reposición



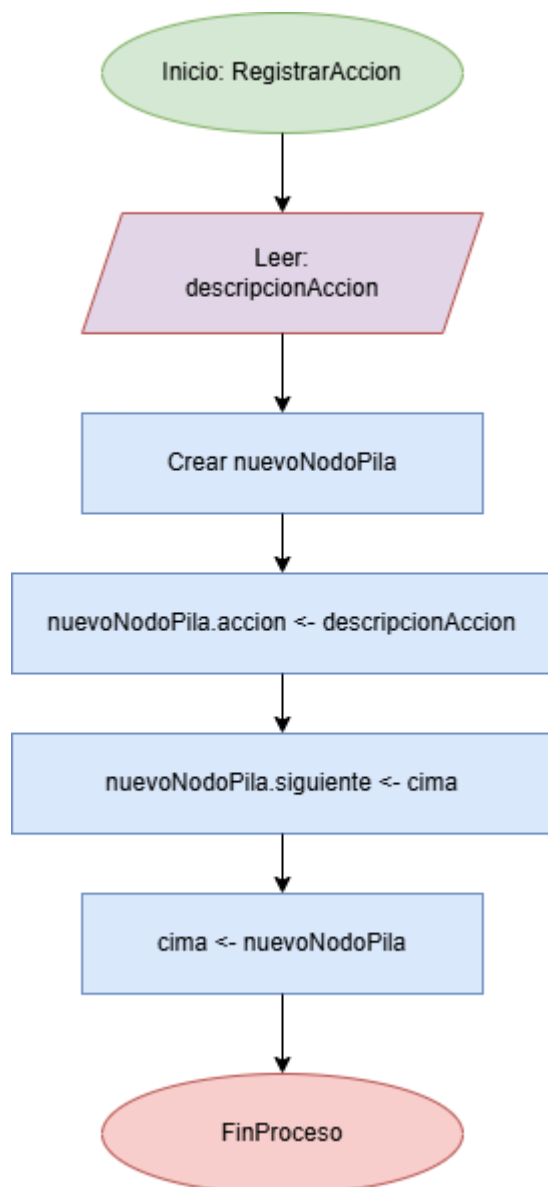
Nota: Diagrama de flujo generado en draw.io

Figura 4. Diagrama de flujo de Atender Reposición



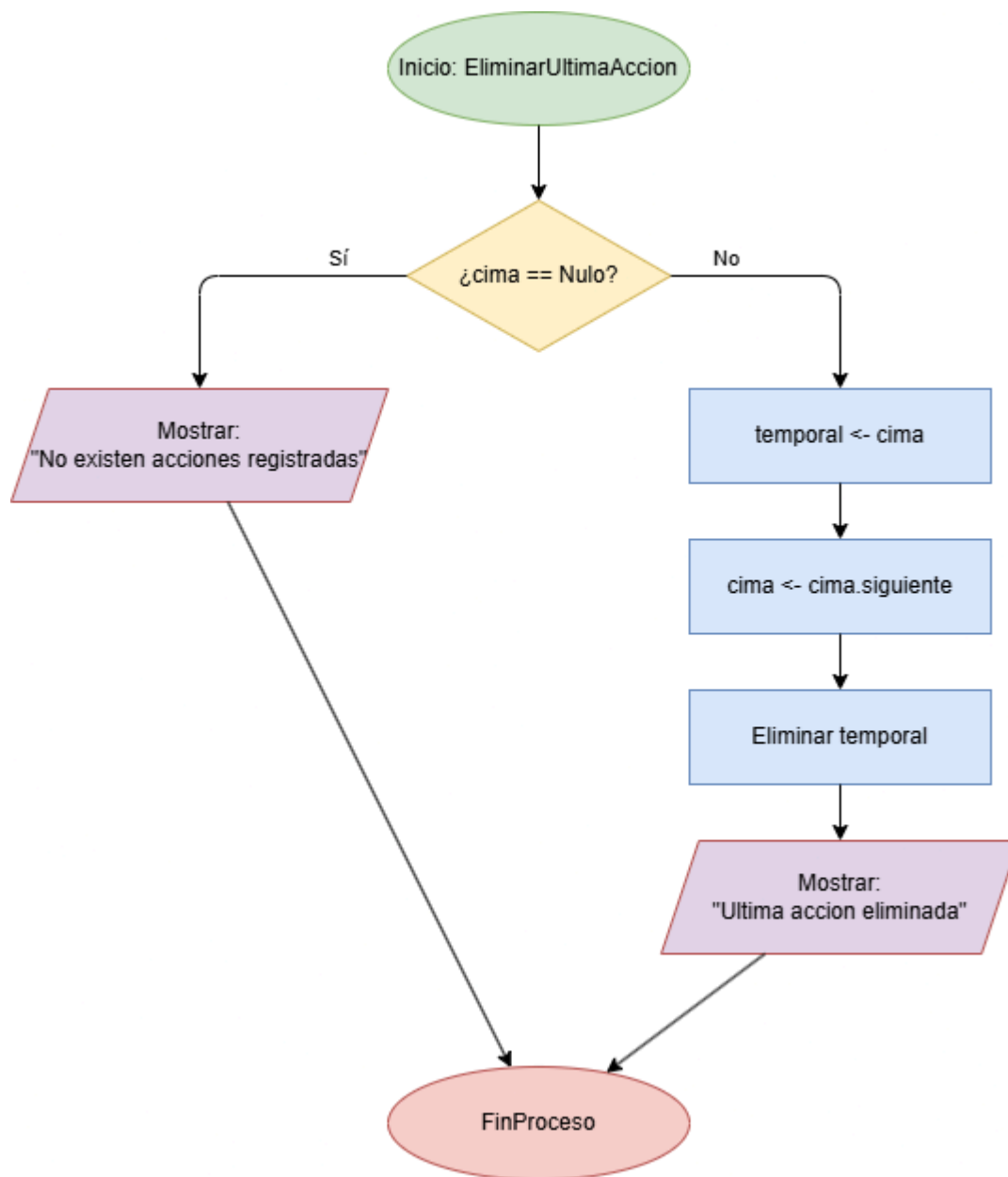
Nota: Diagrama de flujo generado en draw.io

Figura 5. Diagrama de flujo de Registrar Acción



Nota: Diagrama de flujo generado en draw.io

Figura 6. Diagrama de flujo de Eliminar Última Acción



Nota: Diagrama de flujo generado en draw.io

2.4 Justificación del diseño

El diseño propuesto distribuye las responsabilidades del sistema entre distintas estructuras de datos, aprovechando las ventajas de cada una.

La lista enlazada se encarga de la administración principal del inventario, concentrando las operaciones más importantes del sistema.

La cola permite gestionar de forma ordenada los libros que requieren reposición, respetando el orden en que fueron registrados.

La pila facilita el almacenamiento y consulta de acciones recientes realizadas por el usuario.

Además, varias operaciones pueden reutilizar funciones auxiliares, especialmente la búsqueda de libros, reduciendo la repetición de código y facilitando el mantenimiento del sistema.

CAPÍTULO 3: SOLUCIÓN FINAL

3.1 Código limpio y estructurado

En esta sección se presentará la versión final del sistema desarrollado en lenguaje C++. El código estará organizado en funciones y estructuras de datos que faciliten su comprensión y mantenimiento.

Para que el programa funcione correctamente se necesitan seis librerías. Las mas importantes para este proyecto son `fstream`, que hace posible guardar y leer la informacion en archivos de texto para que los datos no se pierdan al cerrar el programa, y `algorithm` junto con `cctype`, que permiten convertir texto a minusculas para que las busquedas de libros funcionen sin importar si el usuario escribe el titulo en mayusculas o no. El resto de librerías cubren la entrada y salida en consola, el manejo de texto con `string` y el control de errores al momento de leer datos.

Figura 7. Imagen Librerías

```
1    #include <iostream>
2    #include <fstream>
3    #include <string>
4    #include <limits>
5    #include <cctype>
6    #include <algorithm>
7    using namespace std;
8
```

Nota: Elaboración propia

Se definieron cuatro structs que representan los bloques de informacion del sistema. El mas importante es Libro, que agrupa los cinco datos que describen a cada libro: titulo, autor, categoria, stock y precio. Los otros tres structs son los nodos de cada estructura de datos: NodoLibro para la lista enlazada, NodoCola para la cola de reposiciones y NodoPila para el historial. Cada nodo guarda su informacion y un puntero llamado siguiente que lo conecta con el proximo nodo, formando la cadena que hace posible recorrer cada estructura.

Figura 8. Estructuras

```
9  ✓ struct Libro {
10      string titulo;
11      string autor;
12      string categoria;
13      int stock;
14      double precio;
15  };
16
17  struct NodoLibro {
18      Libro dato;
19      NodoLibro* siguiente;
20  };
21
22  struct NodoCola {
23      string titulo;
24      NodoCola* siguiente;
25  };
26
27  struct NodoPila {
28      string accion;
29      NodoPila* siguiente;
30  };
31
```

Nota: Elaboración propia

Estas cuatro variables son los puntos de entrada a cada estructura de datos del programa. `cabezaLista` apunta al primer nodo de la lista enlazada donde estan todos los libros. `frenteCola` apunta al primer ticket de reposicion pendiente y `finCola` apunta al ultimo, ya que la cola necesita acceso a ambos extremos para poder agregar por un lado y atender por el otro. `topePila` apunta siempre al nodo mas reciente del historial. Todas inician en `nullptr` indicando que las estructuras estan vacias al comienzo del programa.

Figura 9. Variables globales y punteros principales

```
32  NodoLibro* cabezaLista = nullptr;
33  NodoCola* frenteCola  = nullptr;
34  NodoCola* finCola     = nullptr;
35  NodoPila* topePila    = nullptr;
36
37  const int NUM_CATEGORIAS = 9;
```

Nota: Elaboración propia

En lugar de permitir que el usuario escriba la categoria libremente, se definio un arreglo fijo con nueve opciones. Esto evita inconsistencias como registrar el mismo genero escrito de formas distintas en diferentes libros. La constante `NUM_CATEGORIAS` guarda el total de opciones y el arreglo `CATEGORIAS` contiene los nombres. Cuando el usuario elige una categoria solo necesita ingresar el numero correspondiente y el programa asigna el texto correcto de forma automatica.

Figura 10. Categorías predefinidas

```
38  const string CATEGORIAS[NUM_CATEGORIAS] = {
39      "Narrativa",
40      "Poesia",
41      "Infantil y Juvenil",
42      "Historia",
43      "Ciencia y Tecnologia",
44      "Expositivo / Informativo",
45      "Argumentativo",
46      "Educacion",
47      "Otros"
48  };
```

Nota: Elaboración propia

Muestra las nueve categorías disponibles numeradas en pantalla y espera que el usuario ingrese un número entre 1 y 9. Si el número está fuera de ese rango el programa muestra un aviso y vuelve a pedir la opción sin cerrarse. Cuando la opción es válida retorna el nombre de la categoría como texto para guardarlo en el libro.

Figura 11. Código elegir categoría

```
50  string elegirCategoría() {
51      cout << "\nSeleccione una categoría:\n";
52      for (int i = 0; i < NUM_CATEGORIAS; i++) {
53          cout << " " << (i + 1) << ". " << CATEGORIAS[i] << "\n";
54      }
55
56      int opcion;
57      while (true) {
58          cout << "Opcion: ";
59          if (cin >> opcion && opcion >= 1 && opcion <= NUM_CATEGORIAS) {
60              cin.ignore(numeric_limits<streamsize>::max(), '\n');
61              return CATEGORIAS[opcion - 1];
62          }
63          cout << "[!] Opcion invalida. Ingrese un numero entre 1 y " << NUM_CATEGORIAS << ".\n";
64          cin.clear();
65          cin.ignore(numeric_limits<streamsize>::max(), '\n');
66      }
67  }
```

Nota: Elaboración propia

Cuando el usuario escribe algo incorrecto, como letras donde se esperaba un número, el programa entra en un estado de error que bloquea la entrada. Esta función limpia ese estado y descarta todo lo que quedó pendiente en el buffer, dejando el programa listo para recibir un nuevo dato. Se llama en varias partes del código cada vez que se usa cin con números.

Figura 12. Código limpiar buffer

```
69  void limpiarBuffer() {
70      cin.clear();
71      cin.ignore(numeric_limits<streamsize>::max(), '\n');
72  }
73
74  string aMinusculas(const string& texto) {
75      string resultado = texto;
76      transform(resultado.begin(), resultado.end(), resultado.begin(),
77          [](unsigned char c){ return tolower(c); });
78      return resultado;
79  }
```

Nota: Elaboración propia

Recibe un texto y devuelve el mismo texto con todas sus letras convertidas a minúsculas. Usa transform() junto con tolower() para recorrer cada carácter del string y convertirlo. Se usa antes de comprar títulos durante las búsquedas para que escribir El Principito o EL PRINCIPITO encuentre el mismo libro.

Figura 13. Código para minúsculas

```
74  ✓ string aMinusculas(const string& texto) {  
75      string resultado = texto;  
76      transform(resultado.begin(), resultado.end(), resultado.begin(),  
77          [](unsigned char c){ return tolower(c); });  
78      return resultado;  
79  }
```

Nota: Elaboración propia

Recorre cada caracter de un texto usando isdigit() para verificar si todos son digitos. Si encuentra al menos una letra retorna false y el texto es considerado valido. Si todos son numeros retorna true y el texto es rechazado. Se usa para evitar que el titulo o el autor de un libro sean algo como 123 o 4567, lo que no tendria sentido dentro de un inventario de libros.

Figura 14. Solo números o texto válido

```
81  ✓ bool esSoloNumeros(const string& texto) {  
82      if (texto.empty()) return false;  
83      for (char c : texto) {  
84          if (!isdigit(c)) return false;  
85      }  
86      return true;  
87  }
```

Nota: Elaboración propia.

Pide un texto al usuario y aplica dos controles: que no este vacío y que no sea solo números. Mientras el texto no pase ambas validaciones el programa muestra el mensaje de error correspondiente y vuelve a pedir el dato. Se usa para capturar el título y el autor al momento de registrar o modificar un libro.

Figura 15. Pedir texto sin errores

```
89  ✓ string pedirTextoValido(const string& mensaje) {
90      string valor;
91      while (true) {
92          cout << mensaje;
93          getline(cin, valor);
94          if (valor.empty()) {
95              cout << "[!] El campo no puede estar vacío.\n";
96          } else if (esSoloNumeros(valor)) {
97              cout << "[!] El campo no puede contener solo números.\n";
98          } else {
99              return valor;
100          }
101      }
102  }
```

Nota: Elaboración propia.

Pide un número entero y verifica que la lectura haya sido exitosa y que el valor sea mayor o igual a cero. Si el usuario escribe texto o un número negativo muestra un aviso y repite la solicitud. Se usa exclusivamente para el campo stock, ya que no puede haber una cantidad negativa de libros en el inventario.

Figura 16. Pedir stock sin negativos

```
104  ✓ int pedirEnteroNoNegativo(const string& mensaje) {
105      int valor;
106      while (true) {
107          cout << mensaje;
108          if (cin >> valor && valor >= 0) {
109              limpiarBuffer();
110              return valor;
111          }
112          cout << "[!] Valor inválido. Ingrese un número entero mayor o igual a 0.\n";
113          limpiarBuffer();
114      }
115  }
```

Nota: Elaboración propia.

Funciona igual que la funcion anterior pero para numeros decimales y exige que el valor sea estrictamente mayor que cero. Se usa para el campo precio porque un libro no puede tener precio cero ni negativo. Acepta valores como 12.50 o 100.0 y rechaza cualquier otro tipo de entrada.

Figura 17. Pedir precio mayor a cero

```
117  ✓ double pedirDoublePositivo(const string& mensaje) {
118      double valor;
119      while (true) {
120          cout << mensaje;
121          if (cin >> valor && valor > 0) {
122              limpiarBuffer();
123              return valor;
124          }
125          cout << "[!] Valor invalido. Ingrese un numero mayor que 0.\n";
126          limpiarBuffer();
127      }
128  }
```

Nota: Elaboración propia.

Agrega una nueva accion al tope de la pila. Crea un nuevo nodo con la descripcion de la accion, hace que ese nodo apunte al nodo que antes era el tope y luego actualiza topePila para que apunte al nuevo nodo. Esta funcion se llama automaticamente cada vez que el usuario registra, modifica, elimina un libro o gestiona una reposicion.

Figura 18. Agregar al historial

```
130  ✓ void pushHistorial(const string& accion) {
131      NodoPila* nuevo = new NodoPila;
132      nuevo->accion    = accion;
133      nuevo->siguiente = topePila;
134      topePila         = nuevo;
135  }
136
```

Nota: Elaboración propia.

Muestra en pantalla la accion mas reciente sin eliminarla de la pila. Solo lee el contenido del nodo al que apunta topePila y lo imprime. Si topePila es nullptr significa que la pila esta vacia y muestra un mensaje informandolo. Corresponde a la opcion 8 del menu.

Figura 19. Ver la última acción

```
137  void peekHistorial() {
138      if (topePila == nullptr) {
139          cout << "[!] El historial esta vacio.\n";
140          return;
141      }
142      cout << "\nUltima accion registrada:\n";
143      cout << ">> " << topePila->accion << "\n";
144  }
145
```

Nota: Elaboración propia.

Elimina la accion mas reciente de la pila. Guarda una referencia al nodo del tope actual, imprime su contenido, avanza topePila al nodo que estaba debajo y luego libera la memoria del nodo eliminado con delete. Si la pila esta vacia muestra un aviso antes de intentar cualquier operacion. Corresponde a la opcion 9 del menu.

Figura 20. Quitar la ultima acción

```
146  void popHistorial() {
147      if (topePila == nullptr) {
148          cout << "[!] El historial esta vacio. No hay acciones que eliminar.\n";
149          return;
150      }
151      NodoPila* aEliminar = topePila;
152      cout << "Accion eliminada: " << aEliminar->accion << "\n";
153      topePila = topePila->siguiente;
154      delete aEliminar;
155  }
156
```

Nota: Elaboración propia.

Esta linea le indica al compilador que la funcion buscarNodo existe y sera definida mas adelante en el codigo. Es necesaria porque encolarReposicion la utiliza pero esta escrita antes que ella. Sin esta declaracion el compilador no reconoceria la funcion y mostraria un error.

Figura 21. Declaración adelantada de buscarNodo

```
NodoLibro* buscarNodo(const string& titulo);
```

Nota: Elaboración propia.

Registra un nuevo ticket de reposicion al final de la cola. Antes de crear el nodo realiza dos verificaciones: que el libro exista en el inventario usando buscarNodo() y que no haya ya un ticket pendiente para ese mismo titulo recorriendo la cola. Si pasa ambas validaciones crea el nuevo nodo y lo enlaza despues de finCola. Si la cola estaba vacia, frenteCola y finCola apuntan al mismo nodo nuevo. Corresponde a la opcion 6 del menu.

Figura 22. Registrar un ticket de reposición

```
159 void encolarReposicion(const string& titulo) {
160     if (buscarNodo(titulo) == nullptr) {
161         cout << "[!] El libro \"" << titulo << "\" no existe en el inventario.\n";
162         cout << "    Registre el libro primero antes de solicitar su reposicion.\n";
163         return;
164     }
165
166     string tituloBuscado = aMinusculas(titulo);
167     NodoCola* actual = frenteCola;
168     while (actual != nullptr) {
169         if (aMinusculas(actual->titulo) == tituloBuscado) {
170             cout << "[!] Ya existe una reposicion pendiente para \"" << titulo << "\".\n";
171             return;
172         }
173         actual = actual->siguiente;
174     }
175
176     NodoCola* nuevo = new NodoCola;
177     nuevo->titulo = titulo;
178     nuevo->siguiente = nullptr;
179
180     if (finCola == nullptr) {
181         frenteCola = nuevo;
182         finCola = nuevo;
183     } else {
184         finCola->siguiente = nuevo;
185         finCola = nuevo;
186     }
187
188     cout << "[OK] Reposicion registrada para: " << titulo << "\n";
189     pushHistorial("Reposicion registrada: " + titulo);
190 }
```

Nota: Elaboración propia.

Atiende el ticket mas antiguo de la cola, que siempre es el que esta en frenteCola. Guarda una referencia a ese nodo, muestra su titulo como atendido, avanza frenteCola al siguiente nodo y libera la memoria del nodo eliminado. Si despues de eliminar el frente la cola queda vacia tambien actualiza finCola a nullptr. Si la cola ya estaba vacia muestra un aviso. Corresponde a la opcion 7 del menu.

Figura 23. Atender el primer ticket pendiente

```
192 void desencolarReposicion() {
193     if (frenteCola == nullptr) {
194         cout << "[!] No hay reposiciones pendientes.\n";
195         return;
196     }
197
198     NodoCola* aEliminar = frenteCola;
199     cout << "Reposicion atendida: " << aEliminar->titulo << "\n";
200     frenteCola = frenteCola->siguiente;
201
202     if (frenteCola == nullptr) {
203         finCola = nullptr;
204     }
205
206     pushHistorial("Reposicion atendida: " + aEliminar->titulo);
207     delete aEliminar;
208 }
```

Nota: Elaboración propia.

Recorre la lista desde cabezaLista nodo por nodo comparando titulos. Antes de cada comparacion convierte ambos textos a minusculas con aMinusculas() para que la busqueda no distinga entre mayusculas y minusculas. Si encuentra una coincidencia retorna un puntero directo a ese nodo, lo que permite modificar sus datos sin copiarlos. Si recorre toda la lista sin encontrar el titulo retorna nullptr.

Figura 24. Buscar un libro en la lista

```
210 Nodolibro* buscarNodo(const string& titulo) {
211     string tituloBuscado = aMinusculas(titulo);
212     Nodolibro* actual = cabezaLista;
213
214     while (actual != nullptr) {
215         if (aMinusculas(actual->dato.titulo) == tituloBuscado) {
216             return actual;
217         }
218         actual = actual->siguiente;
219     }
220     return nullptr;
221 }
```

Nota: Elaboración propia.

Recibe un struct Libro por referencia constante y muestra sus cinco campos en pantalla con un formato alineado. Es una funcion auxiliar reutilizable que evita repetir el mismo bloque de impresion en registrarLibro(), buscarLibro(), modificarLibro() y mostrarInventario().

Figura 25. Mostrar los datos de un libro

```

223 void mostrarLibro(const Libro& libro) {
224     cout << " Titulo : " << libro.titulo << "\n";
225     cout << " Autor : " << libro.autor << "\n";
226     cout << " Categoria : " << libro.categoria << "\n";
227     cout << " Stock : " << libro.stock << "\n";
228     cout << " Precio : S/ " << libro.precio << "\n";
229 }
230

```

Nota: Elaboración propia.

Pide los datos del nuevo libro uno por uno aplicando las validaciones de cada campo. Primero verifica con buscarNodo() que no exista otro libro con el mismo titulo. Luego crea un nuevo NodoLibro, recorre la lista hasta el ultimo nodo y enlaza el nuevo al final. Si la lista estaba vacia lo asigna directamente como cabezaLista. Al terminar registra la accion en el historial. Corresponde a la opcion 1 del menu.

Figura 26. Agregar un libro al inventario

```

231 void registrarLibro() {
232     cout << "\nREGISTRAR LIBRO\n";
233
234     string titulo = pedirTextoValido("Titulo : ");
235
236     if (buscarNodo(titulo) != nullptr) {
237         cout << "[!] Ya existe un libro con ese titulo.\n";
238         return;
239     }
240
241     Libro nuevo;
242     nuevo.titulo = titulo;
243     nuevo.autor = pedirTextoValido("Autor : ");
244     nuevo.categoria = elegirCategoria();
245     nuevo.stock = pedirEnteroNoNegativo("Stock : ");
246     nuevo.precio = pedirDoublePositivo("Precio : ");
247
248     NodoLibro* nodoNuevo = new NodoLibro;
249     nodoNuevo->dato = nuevo;
250     nodoNuevo->siguiente = nullptr;
251
252     if (cabezaLista == nullptr) {
253         cabezaLista = nodoNuevo;
254     } else {
255         NodoLibro* actual = cabezaLista;
256         while (actual->siguiente != nullptr) {
257             actual = actual->siguiente;
258         }
259         actual->siguiente = nodoNuevo;
260     }
261
262     cout << "[OK] Libro registrado exitosamente.\n";
263     pushHistorial("Libro registrado: " + nuevo.titulo);
264 }
265

```

Nota: Elaboración propia.

Pide el título del libro al usuario y llama a `buscarNodo()`. Si el resultado es `nullptr` muestra un mensaje indicando que no se encontró ninguna coincidencia. Si el nodo es encontrado llama a `mostrarLibro()` para imprimir toda la información en pantalla. La búsqueda funciona sin importar como el usuario escribió el título gracias a la lógica interna de `buscarNodo()`. Corresponde a la opción 2 del menú.

Figura 27. Encontrar un libro por título

```
266  void buscarLibro() {  
267      cout << "\nBUSCAR LIBRO\n";  
268      string titulo = pedirTextoValido("Titulo a buscar: ");  
269  
270      NodoLibro* encontrado = buscarNodo(titulo);  
271  
272      if (encontrado == nullptr) {  
273          cout << "[!] No se encontro ningun libro con ese titulo.\n";  
274          return;  
275      }  
276  
277      cout << "\nLibro encontrado:\n";  
278      mostrarLibro(encontrado->dato);  
279  }
```

Nota: Elaboración propia.

Busca el libro por título con `buscarNodo()`. Si no existe muestra un error. Si existe muestra los datos actuales y pide los nuevos valores para autor, categoría, stock y precio con sus validaciones. Como `buscarNodo()` retorna un puntero directo al nodo, los cambios se aplican sobre la lista sin necesidad de eliminar y volver a insertar el registro. Corresponde a la opción 3 del menú.

Figura 28. Actualizar datos de un libro

```
281  void modificarLibro() {
282      cout << "\nMODIFICAR LIBRO\n";
283      string titulo = pedirTextoValido("Titulo del libro a modificar: ");
284
285      NodoLibro* nodo = buscarNodo(titulo);
286
287      if (nodo == nullptr) {
288          cout << "[!] No se encontro el libro. No se puede modificar.\n";
289          return;
290      }
291
292      cout << "\nDatos actuales:\n";
293      mostrarLibro(nodo->dato);
294      cout << "\nIngrese los nuevos datos:\n";
295
296      nodo->dato.autor      = pedirTextoValido("Nuevo autor      : ");
297      nodo->dato.categoria  = elegirCategoria();
298      nodo->dato.stock      = pedirEnteroNoNegativo("Nuevo stock    : ");
299      nodo->dato.precio     = pedirDoublePositivo("Nuevo precio   : ");
300
301      cout << "[OK] Libro modificado exitosamente.\n";
302      pushHistorial("Libro modificado: " + nodo->dato.titulo);
303  }
```

Nota: Elaboración propia.

Recorre la lista con dos punteros en paralelo: actual que avanza al siguiente nodo en cada iteración y anterior que se queda un paso atrás. Cuando actual apunta al nodo a eliminar, anterior->siguiente se actualiza para apuntar al nodo que venía después, sacándolo de la cadena. Si el nodo a eliminar es la cabeza se actualiza directamente cabezaLista. Antes de confirmar avisa si ese libro tiene un ticket pendiente en la cola. Corresponde a la opción 4 del menú.

Figura 29. Quitar un libro del inventario

```

304
305 void eliminarLibro() {
306     cout << "\nELIMINAR LIBRO\n";
307     string titulo = pedirTextoValido("Titulo del libro a eliminar: ");
308
309     if (cabezaLista == nullptr) {
310         cout << "[!] El inventario esta vacio.\n";
311         return;
312     }
313
314     NodoLibro* actual = cabezaLista;
315     NodoLibro* anterior = nullptr;
316     string tituloBuscado = aMinusculas(titulo);
317
318     while (actual != nullptr) {
319         if (aMinusculas(actual->dato.titulo) == tituloBuscado) {
320             break;
321         }
322         anterior = actual;
323         actual = actual->siguiente;
324     }
325
326     if (actual == nullptr) {
327         cout << "[!] No se encontro el libro. No se puede eliminar.\n";
328         return;
329     }
330
331     string tituloReal = actual->dato.titulo;
332
333     NodoCola* nCola = frenteCola;
334     while (nCola != nullptr) {
335         if (aMinusculas(nCola->titulo) == tituloBuscado) {
336             cout << "[!] Advertencia: este libro tiene una reposicion pendiente en la cola.\n";
337             cout << "    Considere atenderla (opcion 7) antes de eliminar el libro.\n";
338             break;
339         }
340         nCola = nCola->siguiente;
341     }
342
343     cout << "Confirma eliminar \"" << tituloReal << "\"? (s/n): ";
344     string resp;
345     getline(cin, resp);
346     if (resp != "s" && resp != "S") {
347         cout << "Eliminacion cancelada.\n";
348         return;
349     }
350
351     if (anterior == nullptr) {
352         cabezaLista = actual->siguiente;
353     } else {
354         anterior->siguiente = actual->siguiente;
355     }
356
357     pushHistorial("Libro eliminado: " + tituloReal);
358     delete actual;
359     cout << "[OK] Libro \"" << tituloReal << "\" eliminado del inventario.\n";
360 }
361

```

Nota: Elaboración propia.

Recorre toda la lista desde cabezaLista mostrando cada libro numerado con mostrarLibro(). Si un libro tiene stock igual a cero muestra una advertencia invitando al usuario a registrar una reposición. Al finalizar el recorrido imprime el total de títulos registrados y cuantos están sin stock. Corresponde a la opción 5 del menú.

Figura 30. Ver todos los libros registrados

```
362 void mostrarInventario() {
363     cout << "\nINVENTARIO COMPLETO\n";
364
365     if (cabezaLista == nullptr) {
366         cout << "El inventario esta vacio.\n";
367         return;
368     }
369
370     NodoLibro* actual = cabezaLista;
371     int contador = 1;
372     int sinStock = 0;
373
374     while (actual != nullptr) {
375         cout << "\nLibro #" << contador++ << "\n";
376         mostrarLibro(actual->dato);
377         if (actual->dato.stock == 0) {
378             cout << " [!] SIN STOCK - considere registrar una reposicion\n";
379             sinStock++;
380         }
381         actual = actual->siguiente;
382     }
383
384     cout << "\nTotal de titulos: " << (contador - 1) << "\n";
385     if (sinStock > 0) {
386         cout << "[!] " << sinStock << " libro(s) con stock agotado.\n";
387     }
388 }
```

Nota: Elaboración propia.

Abre libros.txt en modo escritura y recorre toda la lista enlazada escribiendo los cinco campos de cada libro en líneas separadas seguidas de un separador con tres guiones. Esta función se llama automáticamente después de registrar, modificar o eliminar un libro, garantizando que cualquier cambio quede guardado de inmediato sin que el usuario tenga que hacer nada.

Figura 31. Guardar el inventario en disco

```
390  void guardarLibros() {
391      ofstream archivo("libros.txt");
392
393      if (!archivo.is_open()) {
394          cout << "[!] Error: no se pudo guardar libros.txt\n";
395          return;
396      }
397
398      NodoLibro* actual = cabezalista;
399      while (actual != nullptr) {
400          archivo << actual->dato.titulo << "\n";
401          archivo << actual->dato.autor << "\n";
402          archivo << actual->dato.categoria << "\n";
403          archivo << actual->dato.stock << "\n";
404          archivo << actual->dato.precio << "\n";
405          archivo << "---\n";
406          actual = actual->siguiente;
407      }
408
409      archivo.close();
410  }
```

Nota: Elaboración propia.

Se ejecuta al inicio del programa antes de mostrar el menu. Intenta abrir libros.txt y si lo encuentra lee bloques de cinco lineas mas el separador, reconstruye cada Libro y lo inserta al final de la lista enlazada. Los campos stock y precio se convierten de texto a numero con stoi() y stod(). Si el archivo no existe el programa inicia con la lista vacia sin mostrar ningun error.

Figura 32. Cargar el inventario al iniciar

```
412 void cargarLibros() {
413     ifstream archivo("libros.txt");
414     if (!archivo.is_open()) return;
415
416     string linea;
417     while (true) {
418         Libro libro;
419
420         if (!getline(archivo, libro.titulo)) break;
421         if (!getline(archivo, libro.autor)) break;
422         if (!getline(archivo, libro.categoria)) break;
423
424         string stockStr, precioStr;
425         if (!getline(archivo, stockStr)) break;
426         if (!getline(archivo, precioStr)) break;
427
428         libro.stock = stoi(stockStr);
429         libro.precio = stod(precioStr);
430
431         getline(archivo, linea);
432
433         NodoLibro* nodoNuevo = new NodoLibro;
434         nodoNuevo->dato = libro;
435         nodoNuevo->siguiente = nullptr;
436
437         if (cabezaLista == nullptr) {
438             cabezaLista = nodoNuevo;
439         } else {
440             NodoLibro* actual = cabezaLista;
441             while (actual->siguiente != nullptr) {
442                 actual = actual->siguiente;
443             }
444             actual->siguiente = nodoNuevo;
445         }
446     }
447
448     archivo.close();
449 }
```

Nota: Elaboración propia.

Imprime en pantalla las diez opciones disponibles del sistema. No recibe parametros ni retorna nada, solo muestra texto. Se llama al inicio de cada iteración del ciclo principal para que el usuario siempre tenga el menú visible antes de ingresar su opción.

Figura 33. Menú del sistema

```
451  void mostrarMenu() {  
452      cout << "\nSISTEMA DE INVENTARIO - LIBRERIA\n";  
453      cout << "1.  Registrar libro\n";  
454      cout << "2.  Buscar libro\n";  
455      cout << "3.  Modificar libro\n";  
456      cout << "4.  Eliminar libro\n";  
457      cout << "5.  Mostrar inventario\n";  
458      cout << "6.  Registrar reposicion\n";  
459      cout << "7.  Atender reposicion\n";  
460      cout << "8.  Mostrar ultima accion\n";  
461      cout << "9.  Eliminar ultima accion\n";  
462      cout << "10. Salir\n";  
463      cout << "Seleccione una opcion: ";  
464  }  
465
```

Nota: Elaboración propia.

Es el punto de entrada del programa. Llama primero a cargarLibros() para recuperar los datos guardados. Luego ejecuta un ciclo do-while que se repite hasta que el usuario elige la opción 10. En cada iteración muestra el menú, lee la opción del usuario y usa un switch para llamar a la función que corresponde. Al finalizar el ciclo recorre las tres estructuras liberando la memoria de cada nodo con delete para evitar fugas de memoria antes de cerrar el programa.

Figura 34. Función principal del programa

```

000 int main() {
001     cargarLibros();
002
003     int opcion;
004
005     do {
006         mostrarMenu();
007
008         if (!leer la opcion) {
009             cout << "[!] Opcion invalida. Ingrese un numero del 1 al 10.\n";
010             limpiarBuffer();
011             continue;
012         }
013         limpiarBuffer();
014
015         switch (opcion) {
016             case 1:
017                 registrarLibro();
018                 guardarLibros();
019                 break;
020             case 2:
021                 buscarLibro();
022                 break;
023             case 3:
024                 modificarLibro();
025                 guardarLibros();
026                 break;
027             case 4:
028                 eliminarLibro();
029                 guardarLibros();
030                 break;
031             case 5:
032                 mostrarInventario();
033                 break;
034             case 6: {
035                 cout << "\nSELECCIONAR EXPOSICION\n";
036                 string titulo = pedirIntencionalidad("Titulo del libro a registrar: ");
037                 seleccionarExposicion(titulo);
038                 break;
039             }
040             case 7:
041                 cout << "\nSELECCIONAR EXPOSICION\n";
042                 seleccionarExposicion();
043                 break;
044             case 8:
045                 cout << "\nSELECCIONAR ACCION\n";
046                 seleccionarAccion();
047                 break;
048             case 9:
049                 cout << "\nSELECCIONAR ACCION\n";
050                 seleccionarAccion();
051                 break;
052             case 10:
053                 cout << "\nGuardando al sistema. Hasta pronto.\n";
054                 break;
055             default:
056                 cout << "[!] Opcion no valida. Elija entre 1 y 10.\n";
057                 break;
058         }
059     } while (opcion != 10);
060
061     while (cabeceraLista != nullptr) {
062         NodoLibro* temp = cabeceraLista;
063         cabeceraLista = cabeceraLista->siguiente;
064         delete temp;
065     }
066
067     while (frenteCola != nullptr) {
068         NodoCola* temp = frenteCola;
069         frenteCola = frenteCola->siguiente;
070         delete temp;
071     }
072
073     while (topePila != nullptr) {
074         NodoPila* temp = topePila;
075         topePila = topePila->siguiente;
076         delete temp;
077     }
078
079     return 0;
080 }

```

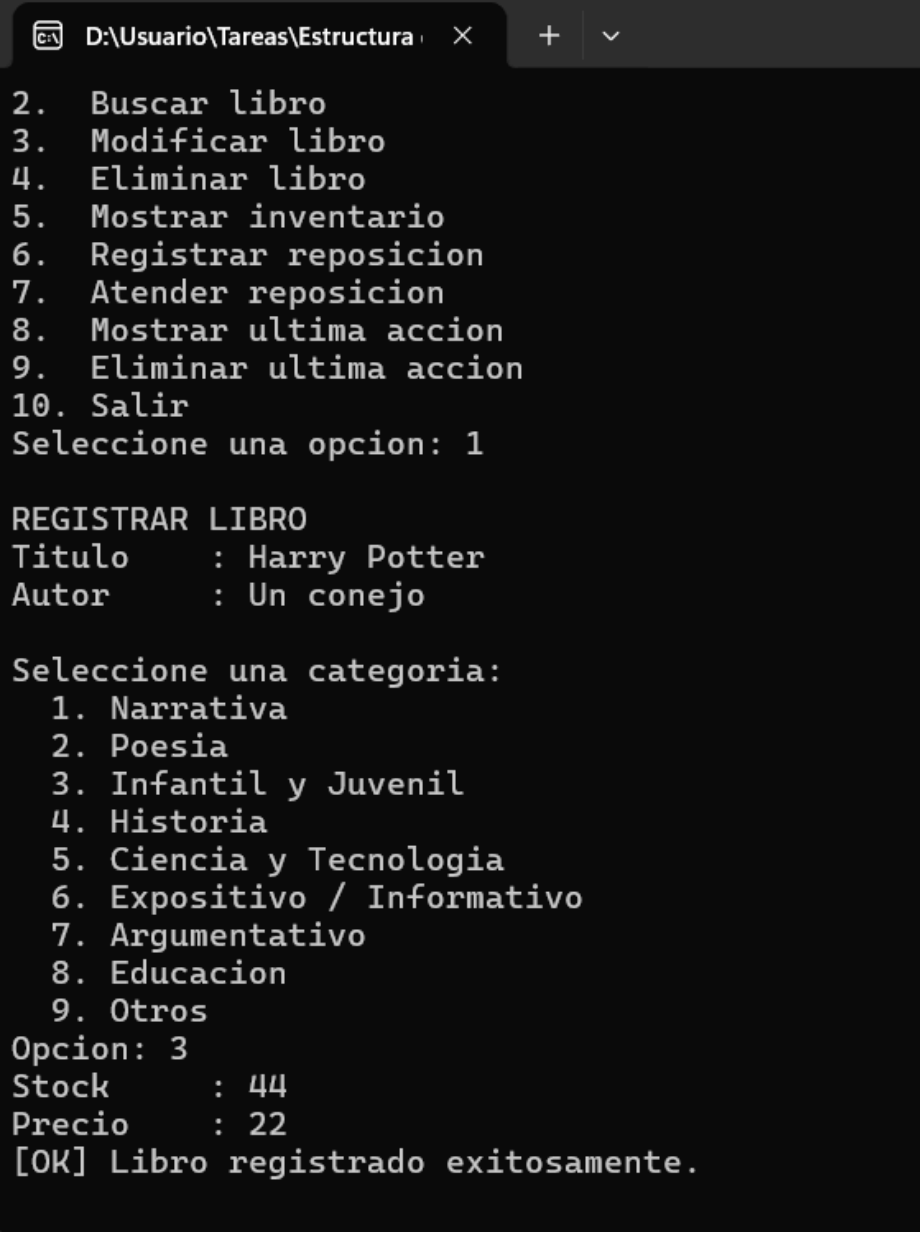
Nota: Elaboración propia.

3.2 Capturas de pantalla de las pruebas realizadas

En esta sección se presentarán evidencias del funcionamiento correcto del sistema mediante capturas de pantalla obtenidas durante la ejecución del programa.

Las capturas deberán mostrar tanto casos exitosos como validaciones implementadas para evitar errores en el ingreso de información.

Figura 35. Registro exitoso de un libro



```
D:\Usuario\Tareas\Estructura > 2. Buscar libro
3. Modificar libro
4. Eliminar libro
5. Mostrar inventario
6. Registrar reposicion
7. Atender reposicion
8. Mostrar ultima accion
9. Eliminar ultima accion
10. Salir
Seleccione una opcion: 1

REGISTRAR LIBRO
Titulo      : Harry Potter
Autor       : Un conejo

Seleccione una categoria:
1. Narrativa
2. Poesia
3. Infantil y Juvenil
4. Historia
5. Ciencia y Tecnologia
6. Expositivo / Informativo
7. Argumentativo
8. Educacion
9. Otros
Opcion: 3
Stock       : 44
Precio      : 22
[OK] Libro registrado exitosamente.
```

Fuente: Elaboración propia.

Figura 36. Búsqueda de un libro registrado

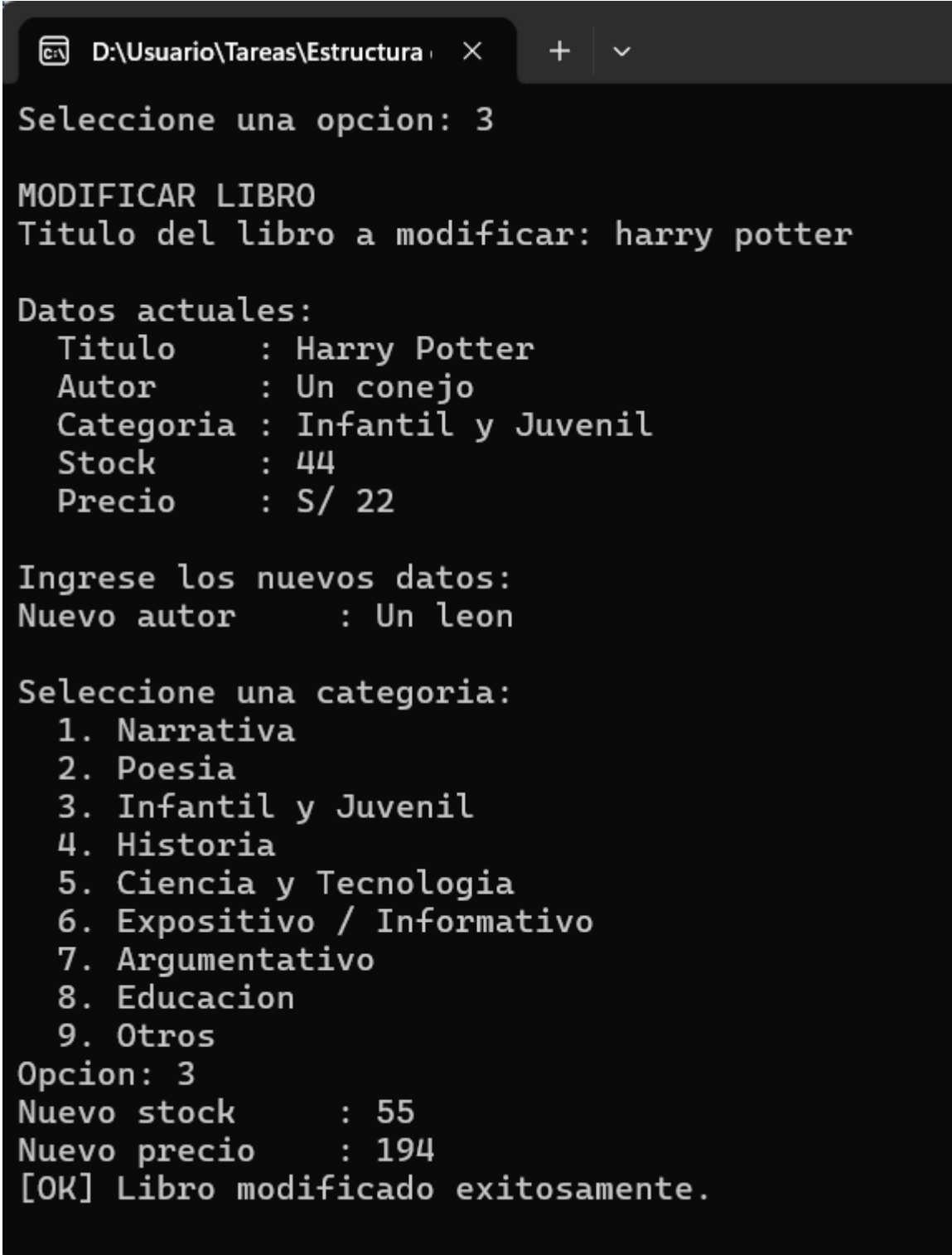
```
SISTEMA DE INVENTARIO - LIBRERIA
1.  Registrar libro
2.  Buscar libro
3.  Modificar libro
4.  Eliminar libro
5.  Mostrar inventario
6.  Registrar reposicion
7.  Atender reposicion
8.  Mostrar ultima accion
9.  Eliminar ultima accion
10. Salir
Seleccione una opcion: 2

BUSCAR LIBRO
Titulo a buscar: harry potter

Libro encontrado:
  Titulo      : Harry Potter
  Autor       : Un conejo
  Categoria   : Infantil y Juvenil
  Stock       : 44
  Precio      : S/ 22
```

Fuente: Elaboración propia.

Figura 37. Modificación de información de un libro



```
D:\Usuario\Tareas\Estructura >
Seleccione una opcion: 3

MODIFICAR LIBRO
Titulo del libro a modificar: harry potter

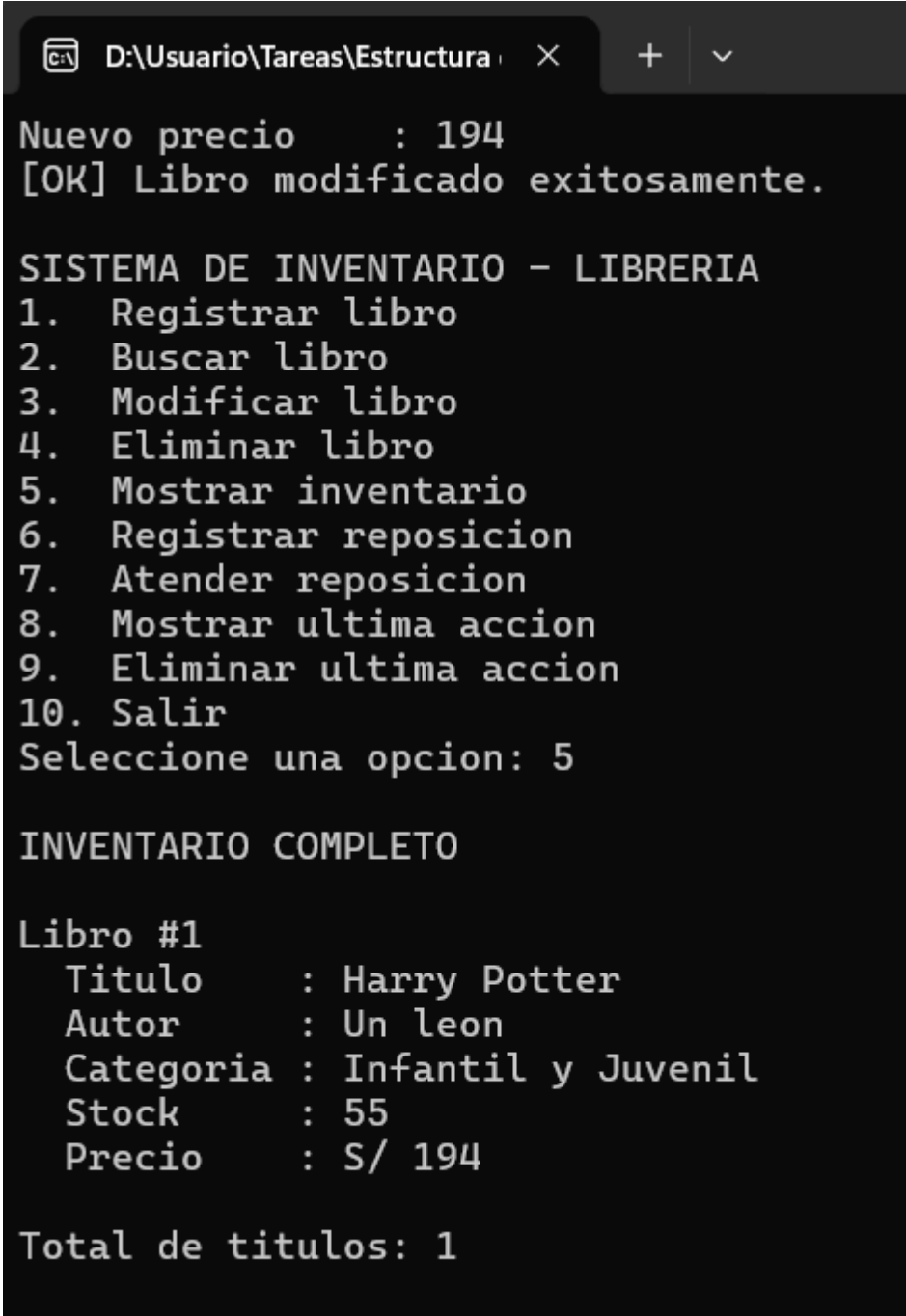
Datos actuales:
Titulo      : Harry Potter
Autor       : Un conejo
Categoria   : Infantil y Juvenil
Stock       : 44
Precio      : S/ 22

Ingrese los nuevos datos:
Nuevo autor : Un leon

Seleccione una categoria:
1. Narrativa
2. Poesia
3. Infantil y Juvenil
4. Historia
5. Ciencia y Tecnologia
6. Expositivo / Informativo
7. Argumentativo
8. Educacion
9. Otros
Opcion: 3
Nuevo stock      : 55
Nuevo precio     : 194
[OK] Libro modificado exitosamente.
```

Fuente: Elaboración propia.

Figura 38. Visualización del inventario



```
D:\Usuario\Tareas\Estructura >
Nuevo precio      : 194
[OK] Libro modificado exitosamente.

SISTEMA DE INVENTARIO - LIBRERIA
1.  Registrar libro
2.  Buscar libro
3.  Modificar libro
4.  Eliminar libro
5.  Mostrar inventario
6.  Registrar reposicion
7.  Atender reposicion
8.  Mostrar ultima accion
9.  Eliminar ultima accion
10. Salir
Seleccione una opcion: 5

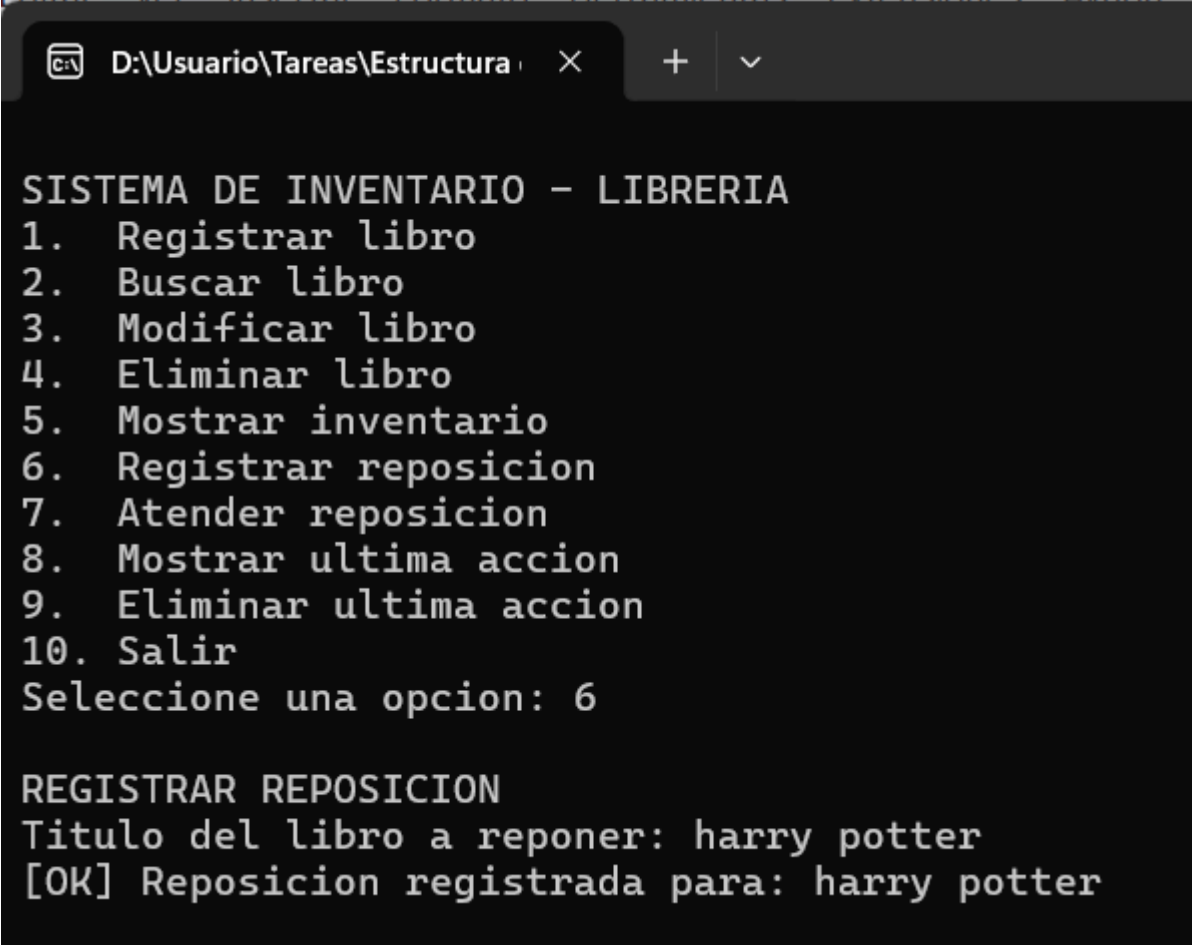
INVENTARIO COMPLETO

Libro #1
  Titulo      : Harry Potter
  Autor       : Un leon
  Categoria   : Infantil y Juvenil
  Stock       : 55
  Precio      : S/ 194

Total de titulos: 1
```

Fuente: Elaboración propia.

Figura 39. Registro de solicitud de reposición

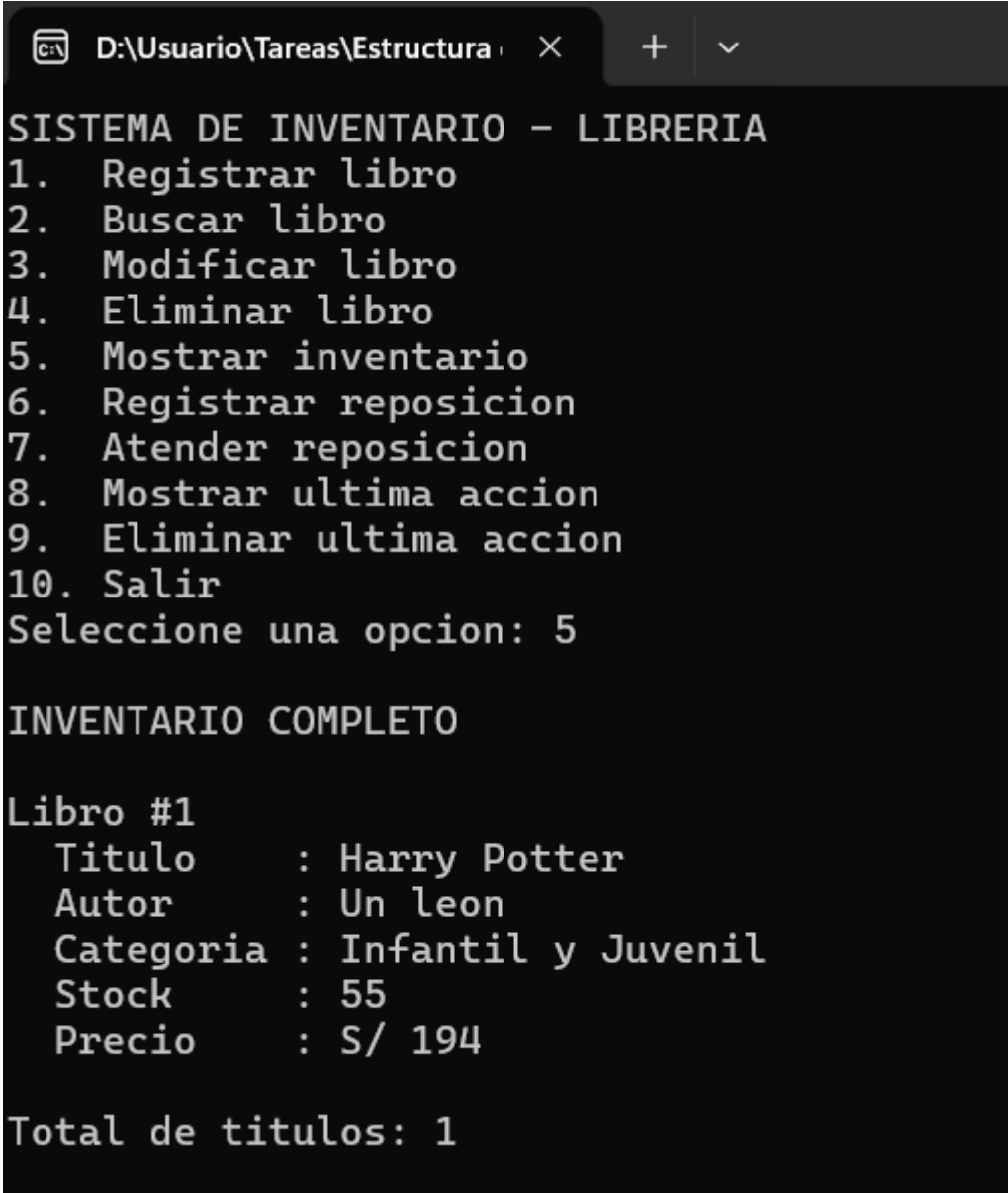


```
D:\Usuario\Tareas\Estructura >
SYSTEMA DE INVENTARIO - LIBRERIA
1.  Registrar libro
2.  Buscar libro
3.  Modificar libro
4.  Eliminar libro
5.  Mostrar inventario
6.  Registrar reposicion
7.  Atender reposicion
8.  Mostrar ultima accion
9.  Eliminar ultima accion
10. Salir
Seleccione una opcion: 6

REGISTRAR REPOSICION
Titulo del libro a reponer: harry potter
[OK] Reposicion registrada para: harry potter
```

Fuente: Elaboración propia.

Figura 40. Consulta del historial de acciones



```
D:\Usuario\Tareas\Estructura >
SISTEMA DE INVENTARIO - LIBRERIA
1.  Registrar libro
2.  Buscar libro
3.  Modificar libro
4.  Eliminar libro
5.  Mostrar inventario
6.  Registrar reposicion
7.  Atender reposicion
8.  Mostrar ultima accion
9.  Eliminar ultima accion
10. Salir
Seleccione una opcion: 5

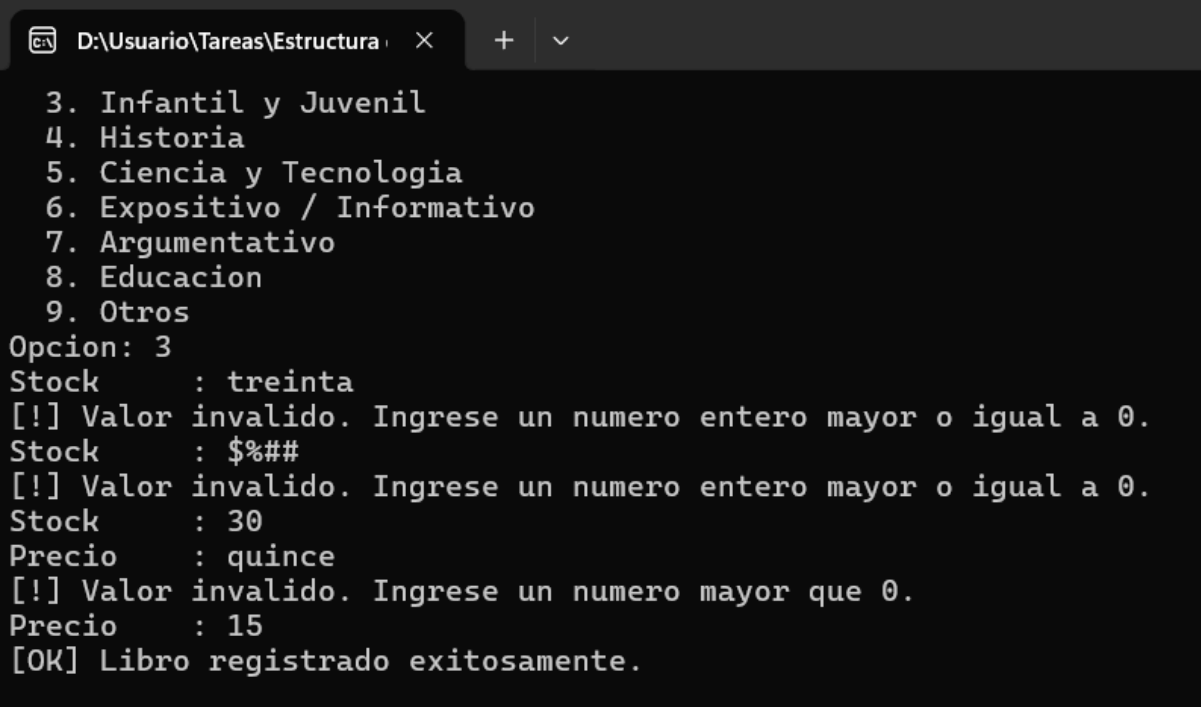
INVENTARIO COMPLETO

Libro #1
  Titulo      : Harry Potter
  Autor       : Un leon
  Categoria   : Infantil y Juvenil
  Stock       : 55
  Precio      : S/ 194

Total de titulos: 1
```

Fuente: Elaboración propia.

Figura 41. Validación de datos incorrectos



```
D:\Usuario\Tareas\Estructura >
3. Infantil y Juvenil
4. Historia
5. Ciencia y Tecnologia
6. Expositivo / Informativo
7. Argumentativo
8. Educacion
9. Otros
Opcion: 3
Stock      : treinta
[!] Valor invalido. Ingrese un numero entero mayor o igual a 0.
Stock      : $%##
[!] Valor invalido. Ingrese un numero entero mayor o igual a 0.
Stock      : 30
Precio     : quince
[!] Valor invalido. Ingrese un numero mayor que 0.
Precio     : 15
[OK] Libro registrado exitosamente.
```

Fuente: Elaboración propia.

3.3 Manual de usuario

Introducción

El presente manual tiene como finalidad explicar el funcionamiento del Sistema de Gestión de Inventario para una Librería. Está dirigido a usuarios sin conocimientos de programación, por lo que cada procedimiento se describe paso a paso de manera sencilla.

El sistema permite registrar libros, buscar información, modificar datos, eliminar registros, gestionar solicitudes de reposición y consultar el historial de acciones realizadas.

Además, cuenta con almacenamiento permanente de datos, por lo que la información registrada no se pierde al cerrar el programa.

Requisitos para ejecutar el sistema

Para utilizar el sistema se requiere:

- Un equipo con sistema operativo Windows.
- El archivo ejecutable del programa llamado **inventario_libreria-0-1.exe**.
- Los archivos generados por el sistema para almacenar la información.
- Acceso a la consola o ventana de ejecución del programa.

Archivos utilizados por el sistema

El sistema utiliza los siguientes archivos:

inventario_libreria-0-1.exe

Es el programa principal que permite ejecutar el sistema.

libros.txt

Archivo de almacenamiento donde se guardan los libros registrados.

Importante:

- No modificar manualmente el contenido del archivo.
- No cambiar el nombre del archivo.
- No eliminar el archivo mientras el sistema esté siendo utilizado.

Formas de ejecución

Opción 1: Ejecutar el archivo compilado

1. Ubicar el archivo **inventario_libreria-0-1.exe**.
2. Hacer doble clic sobre el archivo.
3. Esperar la aparición del menú principal.
4. Seleccionar una opción ingresando el número correspondiente.

Opción 2: Ejecutar desde el entorno de desarrollo

1. Abrir el proyecto en el entorno de programación utilizado.
2. Compilar el programa.
3. Ejecutar la aplicación.
4. Esperar la aparición del menú principal.

Menú principal

Al iniciar el sistema se mostrará un menú con las siguientes opciones:

1. Registrar libro.
2. Buscar libro.
3. Modificar libro.
4. Mostrar inventario.
5. Registrar reposición.
6. Atender reposición.
7. Mostrar última acción.
8. Eliminar última acción.
9. Eliminar libro.
10. Salir.

Registro de libros

1. Seleccionar la opción Registrar libro.
2. Ingresar el título del libro.
3. Ingresar el autor.
4. Ingresar la categoría.
5. Ingresar la cantidad disponible en stock.
6. Ingresar el precio.
7. Confirmar el registro.

Si la información es válida, el sistema almacenará el libro en el inventario y mostrará un mensaje de confirmación.

Búsqueda de libros

1. Seleccionar la opción Buscar libro.
2. Ingresar el título del libro.
3. Esperar el resultado de la búsqueda.

Si el libro existe, se mostrará toda la información registrada.

Modificación de libros

1. Seleccionar la opción Modificar libro.
2. Ingresar el título del libro.
3. Introducir los nuevos datos.
4. Confirmar la actualización.

El sistema actualizará la información almacenada.

Visualización del inventario

1. Seleccionar la opción Mostrar inventario.

El sistema mostrará todos los libros registrados junto con su información correspondiente.

Registro de reposición

1. Seleccionar la opción Registrar reposición.
2. Ingresar el título del libro.
3. Confirmar la solicitud.

La reposición quedará almacenada en la cola de reposiciones pendientes.

Atención de reposiciones

1. Seleccionar la opción Atender reposición.

El sistema procesará automáticamente la solicitud más antigua registrada.

Consulta del historial

1. Seleccionar la opción Mostrar última acción.

El sistema mostrará la acción más reciente realizada dentro del programa.

Eliminación de la última acción

1. Seleccionar la opción Eliminar última acción.

El sistema eliminará el último registro almacenado en el historial.

Eliminación de libros

1. Seleccionar la opción Eliminar libro.
2. Ingresar el título del libro que se desea eliminar.
3. Confirmar la operación.

El sistema eliminará el registro del inventario si el libro existe.

Validación de datos

El sistema verifica automáticamente la información ingresada por el usuario.

Entre las validaciones implementadas se encuentran:

- Campos vacíos.
- Valores negativos para stock.
- Valores negativos para precio.
- Opciones inválidas del menú.
- Búsquedas de libros inexistentes.
- Eliminación de registros inexistentes.

Cuando se detecta un dato incorrecto, el sistema mostrará un mensaje de error y solicitará nuevamente la información.

Almacenamiento automático

Cada vez que se cierre correctamente el sistema mediante la opción Salir, los datos del inventario serán almacenados automáticamente en el archivo de persistencia.

Al volver a ejecutar el programa, la información previamente registrada será cargada nuevamente.

Cierre del sistema

1. Seleccionar la opción Salir.
2. Esperar el mensaje de finalización.

El sistema guardará la información del inventario y finalizará su ejecución correctamente.

CAPÍTULO 4: EVIDENCIAS DE TRABAJO EN EQUIPO

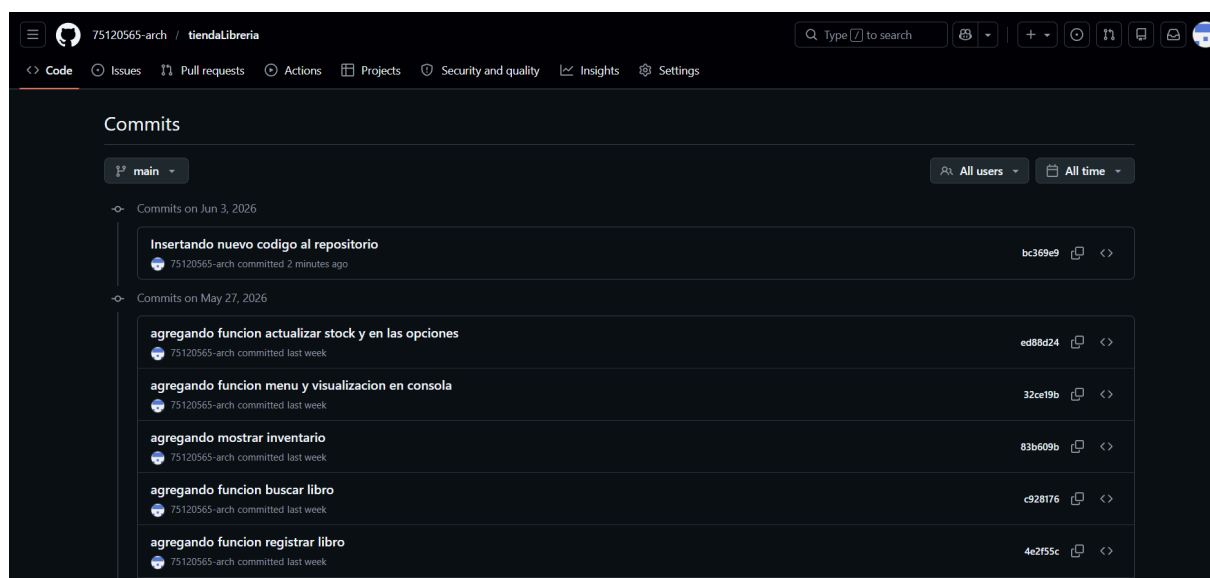
4.1 Repositorio con control de versiones

En esta sección se presentarán las evidencias relacionadas con el trabajo colaborativo realizado durante el desarrollo del proyecto.

Registro de commits

Se incluirán capturas de pantalla donde se observe el historial de commits realizados por los integrantes del equipo.

Figura 42. Historial de commits del proyecto



Fuente: Elaboración propia.

Enlace al repositorio

<https://github.com/75120565-arch/tiendaLibreria>

4.2 Plan de trabajo y roles asignados

Distribución de responsabilidades

Se presentará el documento donde se asignaron las funciones y responsabilidades de cada integrante.

Tabla 3. Distribución de responsabilidades

Integrante	Responsabilidad
Nasdig Ortiz	Análisis del problema
Nasdig Ortiz	Elaboración del informe
Nasdig Ortiz	Diseño de estructuras de datos
Nasdig Ortiz	Desarrollo del código
Nasdig Ortiz	Pruebas del sistema
Nasdig Ortiz	Documentación
Nasdig Ortiz	Elaboración de diagramas de flujo
Nasdig Ortiz	Control de versiones
Nasdig Ortiz	Presentación final

Fuente: Elaboración propia.

Cronograma de actividades

Se presentará la planificación utilizada para organizar el desarrollo del proyecto.

Tabla 4. Cronograma de trabajo

Actividad	Fecha de inicio	Fecha de finalización
Asignación del proyecto y análisis inicial de la problemática	14/05/2026	15/05/2026
Investigación de estructuras de datos aplicables al caso	15/05/2026	17/05/2026
Elaboración de la primera versión del informe	18/05/2026	22/05/2026
Diseño inicial utilizando tablas hash	22/05/2026	27/05/2026
Revisión del avance y observaciones del docente	28/05/2026	28/05/2026
Reestructuración del proyecto utilizando listas, pilas y colas	29/05/2026	30/05/2026
Actualización del informe y corrección de observaciones	30/05/2026	01/06/2026
Diseño de algoritmos y pseudocódigos	01/06/2026	02/06/2026
Elaboración de diagramas de flujo	02/06/2026	02/06/2026
Revisión final del informe y preparación para la entrega	03/06/2026	03/06/2026

Fuente: Elaboración propia.